

Architektur

Unten ist eine detaillierte, code-nahe Beschreibung der HEPA-Architektur in diesem Repository. Die wichtigsten Dateien sind [hepa.py](#), [encoder.py](#), [target_encoder.py](#), [predictor.py](#), [event_head.py](#), [pretrain.py](#) und [finetune.py](#).

Kurzidee

HEPA steht für Horizon-conditioned Event Predictive Architecture. Das Modell lernt zuerst selbstüberwacht aus Zeitreihen, zukünftige Repräsentationen vorherzusagen, nicht direkt zukünftige Rohwerte. Danach wird es für Event-Vorhersage feintrainiert: Für einen aktuellen Zeitpunkt t gibt das Modell für mehrere Horizonte Δt eine monotone Wahrscheinlichkeit aus, ob ein Event innerhalb dieses Horizonts eintritt.

Formal ist das Ziel im Finetuning:

```
p(t, Delta_t_k) = P(Event tritt innerhalb von Delta_t_k Schritten nach t ein)
```

Die Ausgabe ist eine Wahrscheinlichkeitsfläche:

```
p_surface: (B, K)
```

mit B = Batchgröße und K = Anzahl Horizonte.

Gesamtaufbau

Das Modell besteht aus vier Hauptteilen:

```
HEPA
├─ CausalEncoder
│   └─ Kontext  $x[0:t] \rightarrow h_t$ 
├─ TargetEncoder
│   └─ Zukunftsfenster  $x(t:t+\Delta t) \rightarrow h^*$ 
├─ HorizonPredictor
│   └─  $(h_t, \Delta t) \rightarrow$  vorhergesagte Zukunftsrepräsentation  $h_{pred}$ 
└─ EventHead
    └─  $h_{pred} \rightarrow$  Hazard-Logit  $\rightarrow$  monotone CDF  $p(t, \Delta t)$ 
```

Standardkonfiguration laut Code:

```
patch_size      = 16
d_model         = 256
```

```
n_heads      = 4
n_layers     = 2
d_ff         = 256
dropout      = 0.1
predictor_hidden = 256
target_mode  = "joint_train"
norm_mode    = "revin" oder "none"
```

Die Standardarchitektur liegt ungefähr bei 2.16M Parametern.

Datenformat

Die Rohdaten sind multivariate Zeitreihen:

```
x: (T, C)
```

mit:

```
T = Anzahl Zeitpunkte
C = Anzahl Kanäle/Sensoren/Features
```

Ein Dataset-Bundle hat im Code diese Struktur:

```
{
    "pretrain_seqs": Dict[int, np.ndarray], # jede Sequenz: (T_i, C)
    "ft_train": List[Entity],
    "ft_val": List[Entity],
    "ft_test": List[Entity],
    "n_channels": int,
    "horizons": List[int],
    "name": str,
}
```

Eine `Entity` für Finetuning/Evaluation ist:

```
{
    "test": np.ndarray, # (T, C)
    "labels": np.ndarray # (T,), binär: 1 = Event/Anomalie/Zielereignis
}
```

Für das Modell selbst sind die Tensorformen:

```
Pretraining:
context:      (B, T_ctx, C)
target:       (B, T_tgt, C)
delta_t:      (B,)
context_mask: (B, T_ctx), bool, True = Padding
target_mask:  (B, T_tgt), bool, True = Padding
```

```
Finetuning/Inferenz:
context:      (B, T, C)
horizons:     (K,)
context_mask: (B, T), optional
output:       (B, K)
```

Wichtig: Die Masken benutzen `True = Padding`. Gültige echte Werte sind also `False`.

Pretraining-Daten

Im Pretraining wird aus jeder Sequenz mehrfach zufällig ein Schnitt `(t, Delta_t)` gezogen.

Aus einer Sequenz:

```
seq: (T, C)
```

entsteht:

```
context = seq[max(0, t - max_context) : t]
target  = seq[t : t + Delta_t]
delta_t = Delta_t
```

Beispiel:

```
T = 1000
C = 8
max_context = 512
t = 600
Delta_t = 32

context: seq[88:600] -> (512, 8)
target:  seq[600:632] -> (32, 8)
delta_t: 32
```

Die `Delta_t`-Werte werden log-uniform aus einem Bereich gezogen, typischerweise zwischen 1 und dem maximalen Horizont.

Finetuning-/Inferenz-Daten

Im Finetuning wird ein Sliding-Window-Dataset gebaut. Für jeden Zeitpunkt t bekommt das Modell:

```
context = x[max(0, t - max_context) : t]
```

Zusätzlich wird aus den binären Labels die Zeit bis zum nächsten Event berechnet:

```
time_to_event[t] = nächstes_event_index - t
```

Falls kein Event innerhalb max_future kommt:

```
time_to_event[t] = inf
```

Daraus wird die Label-Fläche gebaut:

```
y(t, Delta_t_k) = 1, wenn time_to_event[t] <= Delta_t_k  
y(t, Delta_t_k) = 0, sonst
```

Beispiel:

```
horizons = [1, 5, 10, 20, 50]  
time_to_event[t] = 12  
  
y = [0, 0, 0, 1, 1]
```

Denn das Event kommt nicht innerhalb von 1, 5 oder 10 Schritten, aber innerhalb von 20 und 50.

CausalEncoder

Der `CausalEncoder` verarbeitet nur den Kontext bis Zeitpunkt t . Er darf nicht in die Zukunft schauen. Er erzeugt:

```
h_t: (B, d_model)
```

Ablauf:

```
context x: (B, T, C)  
1. optionale RevIN-Normalisierung  
2. PatchEmbedding
```

3. sinusoidale Positionskodierung
4. kausale Transformer-Blöcke
5. LayerNorm
6. Auswahl des letzten gültigen Tokens

1. RevIN

Bei `norm_mode="revin"` wird pro Batch-Element und Kanal über die Zeit normalisiert:

```
mean[b, :, c] = Mittelwert über gültige Zeitpunkte
std[b, :, c] = Standardabweichung über gültige Zeitpunkte

x_norm = (x - mean) / std
```

Formen:

```
x:      (B, T, C)
mean:   (B, 1, C)
std:    (B, 1, C)
x_norm: (B, T, C)
```

Für C-MAPSS wird laut Konfiguration `norm_mode="none"` verwendet, weil dort absolute Degradationslevel wichtig sind und RevIN diese Information entfernen würde.

2. PatchEmbedding

Zeitpunkte werden in Patches der Länge `patch_size = 16` gruppiert.

```
x: (B, T, C)
P = 16
N = ceil(T / P)
```

Falls `T` nicht durch `P` teilbar ist, wird rechts mit Nullen gepaddet.

Dann:

```
x -> (B, N, C * P)
Linear(C * P -> d_model)
tokens -> (B, N, d_model)
```

Beispiel:

```
B = 4
T = 256
```

```
C = 8
P = 16
N = 16
```

```
context:      (4, 256, 8)
patches:      (4, 16, 128)
after linear: (4, 16, 256)
```

3. Positionskodierung

Zu jedem Patch-Token wird eine sinusoidale Positionskodierung addiert:

```
token_i = token_i + PE(i)
```

Die Form bleibt:

```
(B, N, d_model)
```

Die Sinus/Cosinus-Kodierung entspricht dem klassischen Transformer-Schema:

```
PE(pos, 2j)  = sin(pos / 10000^(2j / d_model))
PE(pos, 2j+1) = cos(pos / 10000^(2j / d_model))
```

4. Kausaler Transformer

Der Kontextencoder nutzt `n_layers = 2` Transformer-Blöcke. Jeder Block ist ein Pre-Norm-Block:

```
x2 = LayerNorm(x)
a  = MultiHeadAttention(x2, x2, x2, causal_mask, padding_mask)
x  = x + Dropout(a)

x2 = LayerNorm(x)
ff = Linear(d_model -> d_ff)
ff = GELU(ff)
ff = Dropout(ff)
ff = Linear(d_ff -> d_model)
ff = Dropout(ff)
x  = x + ff
```

Mit Standardwerten:

```
d_model = 256
n_heads = 4
```

```
d_ff = 256
```

Die kausale Maske verhindert, dass Patch i auf spätere Patches $j > i$ schaut.

5. Ausgabe

Nach allen Blöcken kommt eine finale `LayerNorm`. Dann wird der letzte gültige Patch-Token genommen:

```
h_t = h[:, -1]
```

oder bei Padding der letzte nicht-gepadete Patch.

Ergebnis:

```
h_t: (B, 256)
```

TargetEncoder

Der `TargetEncoder` verarbeitet das Zukunftsfenster:

```
target = x(t : t + Delta_t]
```

Er sieht also den Zielabschnitt vollständig und bidirektional. Er wird nur im Pretraining verwendet.

Ablauf:

```
target x: (B, T_tgt, C)
1. optionale RevIN-Normalisierung
2. PatchEmbedding
3. sinusoidale Positionskodierung
4. nicht-kausale Transformer-Blöcke
5. LayerNorm
6. Attention Pooling mit lernbarer Query
```

Der große Unterschied zum Kontextencoder: keine kausale Maske. Innerhalb des Target-Fensters darf jeder Patch jeden anderen Patch sehen.

Nach den Transformer-Blöcken gibt es ein Attention-Pooling:

```
query = learned_pool_query: (1, 1, d_model)
query wird auf Batch expandiert: (B, 1, d_model)
```

```
pooled = MultiHeadAttention(query, h, h)
h_target = pooled.squeeze(1)
```

Ergebnis:

```
h_target: (B, 256)
```

HorizonPredictor

Der Predictor bekommt den Kontextzustand `h_t` und den Horizont `Delta_t`.

Input:

```
h_t:      (B, 256)
delta_t: (B,)
```

Zuerst wird `delta_t` angehängt:

```
dt = delta_t.unsqueeze(-1)    -> (B, 1)
z  = concat(h_t, dt)         -> (B, 257)
```

Dann läuft ein MLP:

```
Linear(257 -> 256)
GELU
Linear(256 -> 256)
GELU
Linear(256 -> 256)
```

Output:

```
h_pred: (B, 256)
```

Interpretation:

```
h_pred = vorhergesagte Repräsentation des zukünftigen Fensters
        für den konkreten Horizont Delta_t
```

Pretraining-Loss

Im Pretraining soll `h_pred` nahe an `h_target` liegen. Der Loss ist:


```
L = (1 - alpha) * L1(normalize(h_pred), normalize(h_target))  
    + alpha * (L_var + L_cov)
```

Standard:

```
alpha = 0.1
```

Der Alignment-Term nutzt L2-normalisierte Vektoren:

```
pred_n = h_pred / ||h_pred||_2  
targ_n = h_target / ||h_target||_2  
  
L1 = mean(abs(pred_n - targ_n))
```

Der Regularizer ist VICReg-artig und wird auf dem rohen `h_pred` berechnet.

Varianz-Term:

```
std_j = std(h_pred[:, j])  
L_var = mean_j ReLU(1 - std_j)
```

Das verhindert, dass alle Features kollabieren und fast konstant werden.

Kovarianz-Term:

```
h_c = h_pred - mean(h_pred)  
cov = h_c.T @ h_c / (B - 1)  
L_cov = Summe der quadrierten Off-Diagonal-Einträge / D
```

Das drückt verschiedene Feature-Dimensionen auseinander, damit sie nicht alle dieselbe Information tragen.

Target-Encoder-Modi

Das Modell kennt drei Modi:

```
joint_train      = Default  
periodic_sync    = Ablation  
frozen_target    = Ablation
```

Bei `joint_train` erhalten Kontextencoder und TargetEncoder Gradienten. Der TargetEncoder wird initial aus dem Encoder kopiert, ist danach aber trainierbar.

Bei `periodic_sync` wird der TargetEncoder nicht per Gradient trainiert, sondern alle `sync_interval_steps` hart vom Encoder kopiert.

Bei `frozen_target` bleibt der TargetEncoder nach der Initialisierung eingefroren.

Finetuning

Nach dem Pretraining wird für Event Prediction feintrainiert. Standardmodus:

```
mode = "pred_ft"
```

Dabei wird der Encoder eingefroren:

```
encoder.requires_grad = False
```

Trainiert werden nur:

```
predictor  
event_head
```

Das ist ein zentrales Design: Der Encoder enthält die selbstüberwacht gelernte Zeitreihenrepräsentation; der Predictor wird auf das konkrete Event-Ziel angepasst.

EventHead und monotone CDF

Für jeden Horizont `Delta_t_k` wird ein vorhergesagtes Embedding erzeugt:

```
h_pred_k = predictor(h_t, Delta_t_k)
```

Dann:

```
logit_k = event_head(h_pred_k)  
lambda_k = sigmoid(logit_k)
```

`lambda_k` ist eine diskrete Hazard-Wahrscheinlichkeit. Danach wird daraus eine Survival-Funktion gebildet:

```
S_k = Produkt_{j <= k} (1 - lambda_j)
```

Und daraus die kumulative Event-Wahrscheinlichkeit:

```
p(t, Delta_t_k) = 1 - S_k
```

Im Code:

```
lambdas = torch.sigmoid(hazard_logits)
survival = torch.cumprod(1 - lambdas, dim=-1)
cdf = 1 - survival
```

Dadurch ist die Ausgabe entlang der Horizonte automatisch monoton nicht-fallend:

```
p(t, Delta_t_1) <= p(t, Delta_t_2) <= ... <= p(t, Delta_t_K)
```

Das ist sinnvoll, weil die Wahrscheinlichkeit, dass ein Event innerhalb von 50 Schritten passiert, nicht kleiner sein sollte als die Wahrscheinlichkeit innerhalb von 10 Schritten.

Finetuning-Loss

Die Ausgabe ist:

```
cdf: (B, K)
```

Die Labels sind:

```
y: (B, K)
```

Der Loss ist positive-weighted BCE:

```
L = -mean(pos_weight * y * log(p) + (1 - y) * log(1 - p))
```

`pos_weight` gleicht seltene Events aus.

Inferenz

Für Inferenz braucht man keinen TargetEncoder. Man braucht nur:

```
context
horizons
encoder
predictor
event_head
```

Ablauf:

1. Kontext bis Zeitpunkt `t` vorbereiten: `(B, T, C)`
2. Encoder berechnet `h_t`: `(B, 256)`

3. h_t wird für alle K Horizonte kopiert: $(B \times K, 256)$
4. Horizonte werden ebenfalls expandiert: $(B \times K,)$
5. Predictor berechnet h_{pred} für jedes Paar (Sample, Horizont)
6. EventHead erzeugt Hazard-Logits: (B, K)
7. Sigmoid $\rightarrow$ Hazards
8. Cumprod $\rightarrow$ Survival
9. $1 - \text{Survival} \rightarrow$ monotone CDF

Output:

```
p_surface: (B, K)
```

Komplettes Beispiel

Nehmen wir ein kleines Beispiel nahe am Testcode:

```
B = 4
T_ctx = 256
T_tgt = 32
C = 8
d_model = 256
patch_size = 16
horizons = [1, 5, 10, 20, 50]
```

Pretraining-Input:

```
context: (4, 256, 8)
target:  (4, 32, 8)
delta_t: [16, 32, 24, 8]
```

Schritt 1: Kontextencoder.

```
context: (4, 256, 8)
RevIN:   (4, 256, 8)
Patches: 256 / 16 = 16 Tokens
reshape: (4, 16, 128)
Linear:  (4, 16, 256)
+ PE:    (4, 16, 256)
Transformer Block 1: (4, 16, 256)
Transformer Block 2: (4, 16, 256)
LayerNorm:           (4, 16, 256)
last token:           (4, 256)
```

Also:

```
h_t: (4, 256)
```

Schritt 2: Predictor.

```
h_t:      (4, 256)
delta_t: (4,) -> (4, 1)
concat:   (4, 257)

MLP:
257 -> 256 -> 256 -> 256
```

Also:

```
h_pred: (4, 256)
```

Schritt 3: TargetEncoder.

```
target: (4, 32, 8)
RevIN:  (4, 32, 8)
Patches: 32 / 16 = 2 Tokens
reshape: (4, 2, 128)
Linear:  (4, 2, 256)
+ PE:    (4, 2, 256)
Transformer Block 1 ohne causal mask: (4, 2, 256)
Transformer Block 2 ohne causal mask: (4, 2, 256)
LayerNorm: (4, 2, 256)
Attention Pooling: (4, 1, 256)
squeeze: (4, 256)
```

Also:

```
h_target: (4, 256)
```

Schritt 4: Pretraining-Loss.

```
normalize(h_pred): (4, 256)
normalize(h_target): (4, 256)
L1 alignment
+ variance/covariance regularizer auf h_pred
```

Danach Backpropagation und Optimizer-Step.

Jetzt Finetuning/Inferenz für dieselben Kontextdaten:

```
context: (4, 256, 8)
horizons: (5,) = [1, 5, 10, 20, 50]
```

Schritt 1: Encoder.

```
h_t: (4, 256)
```

Schritt 2: Für alle Horizonte expandieren.

```
h_t expanded:
(4, 5, 256) -> (20, 256)

horizons expanded:
(4, 5) -> (20,)
```

Schritt 3: Predictor für jedes Sample-Horizont-Paar.

```
predictor input: (20, 257)
predictor output: (20, 256)
reshape:         (4, 5, 256)
```

Schritt 4: EventHead.

```
LayerNorm: (4, 5, 256)
Linear:     (4, 5, 1)
squeeze:    (4, 5)
```

Das sind die Hazard-Logits.

Angenommen für ein Sample entstehen nach Sigmoid diese Hazards:

```
lambda = [0.05, 0.10, 0.20, 0.30, 0.40]
```

Dann:

```
S_1 = 1 - 0.05 = 0.95
S_2 = 0.95 * (1 - 0.10) = 0.855
```

```
S_3 = 0.855 * (1 - 0.20) = 0.684  
S_4 = 0.684 * (1 - 0.30) = 0.4788  
S_5 = 0.4788 * (1 - 0.40) = 0.28728
```

CDF:

```
p = 1 - S  
p = [0.05, 0.145, 0.316, 0.5212, 0.71272]
```

Interpretation:

```
P(Event innerhalb 1 Schritt) = 5.0%  
P(Event innerhalb 5 Schritte) = 14.5%  
P(Event innerhalb 10 Schritte)= 31.6%  
P(Event innerhalb 20 Schritte)= 52.1%  
P(Event innerhalb 50 Schritte)= 71.3%
```

Diese Werte sind monoton steigend, weil sie aus einer Survival-Funktion konstruiert werden.

Wichtigste Designentscheidung

HEPA trennt Repräsentationslernen und Event-Anpassung:

```
Pretraining:  
unlabeled Zeitreihen -> lerne  $h_t$ , das zukünftige Dynamik vorhersagen kann  
  
Finetuning:  
labeled Eventdaten -> friere Encoder ein, trainiere Predictor + EventHead
```

Dadurch kann das Modell viel unbeschriftete Zeitreihendynamik nutzen und braucht für seltene Events weniger gelabelte Daten.